AI AGENTS ORCHESTRATOR

Technical Analysis Report

Repository: github.com/hoangsonww/AI-Agents-Orchestrator

Prepared by: Senior AI Systems Architect & Technical Writer

Date: April 2026 | Version 1.0.0 (Public Stable Release)

Classification: Internal — Interview / Management Presentation Ready

Python 3.8+	Vue 3 / Nuxt	Flask + Socket.IO	Docker / K8s	Prometheus
Multi-Agent Orchestration	Agentic Team Runtime	Graph Context Memory	Offline/Local LLM Support	CI/CD Ready

SECTION 01

Executive Summary

The AI Agents Orchestrator (also known as the AI Coding Tools Orchestrator) is a production-ready, open-source platform designed to coordinate multiple AI coding assistants — including Anthropic Claude, OpenAI Codex, Google Gemini CLI, and GitHub Copilot CLI — in a unified, configurable workflow system. The project, authored by Son Nguyen (@hoangsonww), was publicly released as v1.0.0 in November 2025 and has rapidly attracted community interest, achieving 32 GitHub stars and 15 forks within months of release.

At its core, the system solves a critical problem in modern AI-assisted software development: no single AI model excels at every aspect of software engineering. By chaining agents in structured workflows (implement → review → refine), the orchestrator extracts complementary strengths from each model, producing higher-quality outputs than any individual agent could achieve alone.

Key Strengths

- Enterprise-grade architecture with Pylint 10.00/10 score and 386 passing tests
- Dual runtime model: Orchestrator workflows and a fully independent Agentic Team runtime
- Supports both cloud AI CLIs and local LLMs (Ollama, llama.cpp-compatible servers)
- Rich observability: Prometheus metrics, structured logging, HTML dashboards, and report generation
- Graph-based persistent memory system with BM25, semantic, and hybrid search
- Full deployment stack: Docker, Kubernetes manifests, systemd service files, CI/CD pipelines
- Two user interfaces: CLI REPL (Click + Rich) and a Vue 3 / Nuxt web dashboard with Monaco editor

Dimension	Details
Language Composition	Python 57.2%, JavaScript 12.8%, HTML 9.8%, Vue 6.8%, CSS 5.3%, Shell 4.2%
Primary AI Agents	Claude Code CLI, OpenAI Codex CLI, Google Gemini CLI, GitHub Copilot CLI, Ollama, llama.cpp
Code Quality	Pylint 10.00/10, 386 tests, 15/15 pre-commit hooks green, >80% coverage
Deployment Targets	Docker Compose, Kubernetes, systemd, Azure, bare-metal
License	MIT — open source, commercially usable
Release	v1.0.0 — November 29, 2025

SECTION 02

Project Overview

Purpose and Problem Statement

Modern software development increasingly relies on AI coding assistants. However, each model has different strengths: Codex excels at initial code generation, Gemini at code review and analysis, Claude at nuanced reasoning and documentation, and Copilot at inline suggestions. The AI Agents Orchestrator provides a coordination layer that harnesses all of these capabilities in a single, extensible platform.

Two Distinct Runtime Modes

1. Orchestrator Workflow Mode

Agents execute in a configurable sequential pipeline. For example, the 'default' workflow chains Codex (implementation) → Gemini (review) → Claude (refinement). The orchestrator manages context passing, retry logic, fallback routing, and result aggregation.

2. Agentic Team Runtime

A fully separate runtime where AI agents assume named roles (Project Manager, Software Architect, Developer, QA Engineer, DevOps Engineer) and communicate freely turn-by-turn. The lead role gates the final response, ensuring structured output after open deliberation.

User Interfaces

Interface	Technology	Port	Key Feature
CLI REPL (Orchestrator)	Click + Rich	N/A	Interactive shell, follow-up detection, session save/load
CLI REPL (Agentic Team)	Click + Rich	N/A	agentic-shell with --max-turns and --offline flags
Web UI (Orchestrator)	Vue 3 + Vite + Flask	3000/5001	Monaco editor, real-time Socket.IO updates, file download
Web UI (Agentic Team)	Flask + Socket.IO	5002	Live communication graph, turn timeline, runtime logs
Context Dashboard	Flask + vis.js + Chart.js	5003	Interactive graph explorer, analytics, export

Available Workflows

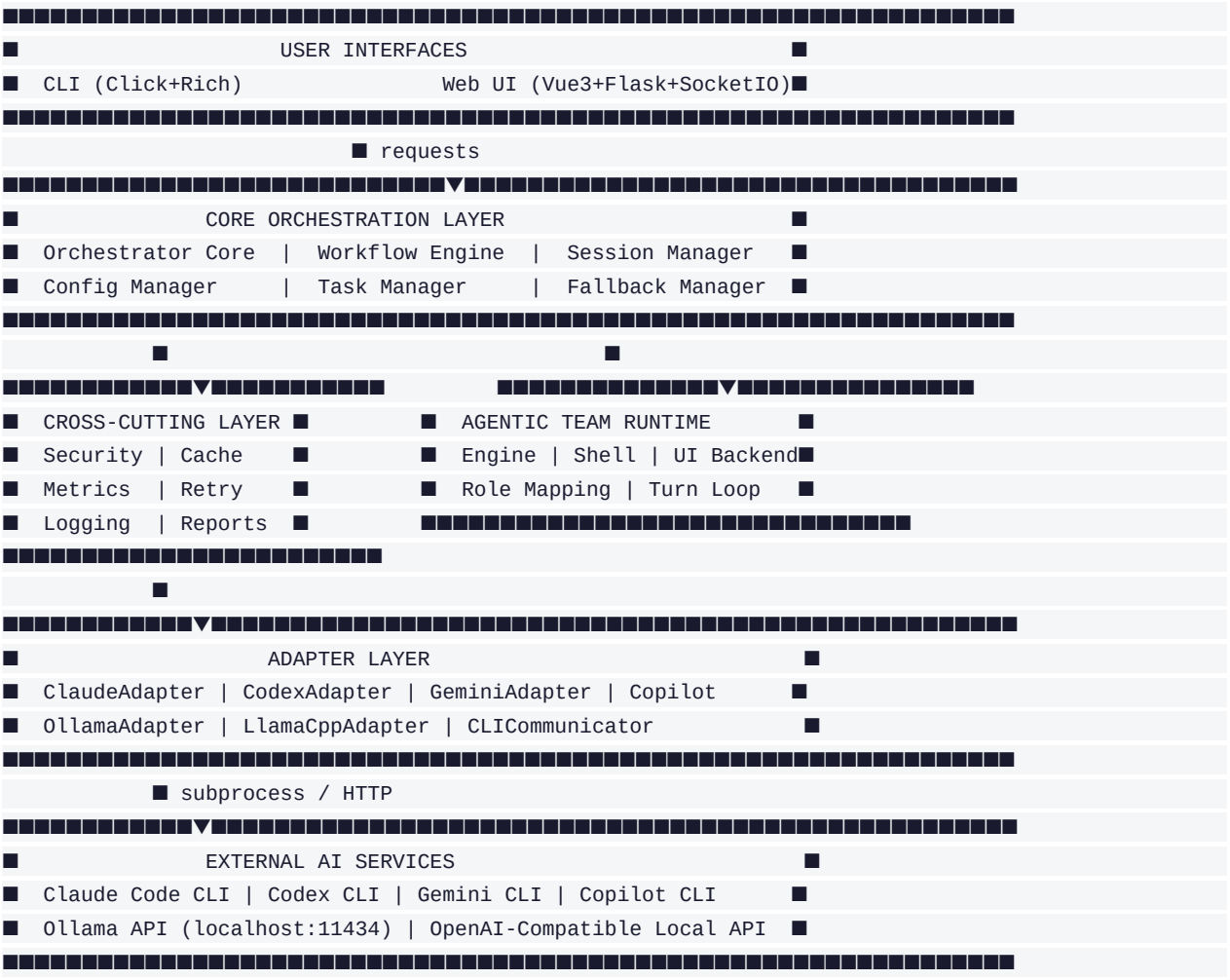
Workflow	Pipeline	Best Use Case
default	Codex → Gemini → Claude	Production-quality code with full review cycle
quick	Codex only	Fast prototyping
thorough	Codex → Copilot → Gemini → Claude → Gemini	Mission-critical / security-sensitive code
review-only	Gemini → Claude	Analyzing and improving existing code
document	Claude → Gemini	Generating comprehensive documentation
offline-default	local-code → local-instruct	No cloud dependency, full local execution

Workflow	Pipeline	Best Use Case
hybrid	local-code → claude (fallback local-instruct)	Local draft with cloud review and auto-fallback

SECTION 03

Architecture Diagram (Text Explanation)

The system follows a layered, hexagonal architecture with clear separation between user interfaces, the orchestration core, cross-cutting concerns, the adapter abstraction layer, runtime controls, and external AI services. The following describes each layer:



■ The Agentic Team runtime is architecturally isolated — it never imports from orchestrator/ and maintains its own context database, adapters, and session state. This boundary is intentional and enforced.

SECTION 04

Detailed Architecture Breakdown

4.1 Interface Layer

The interface layer provides two entry points: the CLI (built with Click and Rich for terminal rendering) and the Web UI (Vue 3 frontend with a Flask/Socket.IO backend). Both interfaces converge on the same orchestration API. The CLI supports a REPL-style shell with readline history, tab completion, and smart follow-up detection. The Web UI adds Monaco editor integration, real-time progress streaming, and file download capabilities.

4.2 Orchestration Core

orchestrator/core.py is the central coordination component. It receives a task and workflow name, validates inputs through the Security layer, resolves offline/online mode via the OfflineDetector, instantiates adapters from agents.yaml configuration, and delegates execution to the Workflow Engine. After execution it aggregates results, updates the session, triggers report generation, and returns the final output.

4.3 Workflow Engine

orchestrator/workflow.py manages workflow definitions and their sequential execution. Each workflow is a list of steps, each binding an agent name to a role (implement, review, refine, document). The engine iterates steps, passes accumulated context from each step to the next, checks stop conditions (maximum iterations, no new suggestions), and supports step-level fallback — if a step's primary agent fails due to a recoverable error, the engine tries the mapped fallback agent.

4.4 Adapter Layer

The adapter layer implements the Adapter design pattern. All six adapters extend BaseAdapter (adapters/base.py), which defines the interface: get_capabilities(), execute_task(), execute_task_async(), and is_available(). Cloud adapters (Claude, Codex, Gemini, Copilot) communicate with external CLI tools via the CLICommunicator subprocess abstraction. Local adapters (Ollama, LlamaCpp) communicate via HTTP to local REST APIs, supporting full async execution with httpx.

4.5 Cross-Cutting Services

Module	File	Responsibility
SecurityManager	orchestrator/security.py	Input validation, command-injection prevention, rate limiting, audit logging
CacheLayer	orchestrator/cache.py	In-memory (TTL 5 min) and file-based (TTL 24h) response caching
RetryLogic	orchestrator/retry.py	Exponential backoff via tenacity decorators on adapter calls
MetricsCollector	orchestrator/metrics.py	Prometheus counters/histograms/gauges exposed on :9090/metrics
ReportGenerator	orchestrator/observability/	JSON + HTML dashboard reports auto-generated per task execution
OfflineDetector	orchestrator/offline.py	Cached connectivity checks; auto-routes to local agents if offline
FallbackManager	orchestrator/fallback.py	Maps cloud agents to local fallbacks; invoked on recoverable failures

Module	File	Responsibility
SessionManager	orchestrator/core.py	Persist/restore conversation context to/from sessions/ directory

4.6 Agentic Team Runtime

The agentic_team/ package is a standalone runtime with its own engine (agentic_team/engine.py), CLI REPL (agentic_team/shell.py), and web backend (ui/agentic_app.py). The engine drives a turn-by-turn loop: at each turn, it sends a structured prompt (task + roster + transcript + incoming message) to the current role's bound adapter, parses the JSON decision response (action: 'message' or 'finalize', to_role, message), routes accordingly, and emits Socket.IO events for real-time UI updates. Execution terminates when the lead role issues a 'finalize' action, or when the max_turns ceiling is reached.

4.7 Graph Context Memory System

Both runtimes maintain persistent knowledge graphs in SQLite databases. The Orchestrator context at ~/.ai-orchestrator/context.db and the Agentic Team context at ~/.agentic-team/context.db. Each graph stores 10 node types (conversation, task, mistake, pattern, decision, code_snippet, preference, file, concept, agent_output) connected by 12 semantic edge types. The Orchestrator context additionally supports sentence-transformer embeddings (all-MiniLM-L6-v2) for semantic search, combined with BM25 keyword search via Reciprocal Rank Fusion. Both systems offer analytics, pruning, export (JSON, GraphML), and import tooling.

SECTION 05

Technology Stack

Backend — Python Core

Library / Tool	Version	Role in System
Python	3.8+	Primary runtime language for all backend logic
Click	8.x	CLI framework — command parsing, argument handling, REPL entry points
Rich	13.x	Terminal formatting — colored output, progress bars, tables, panels
Pydantic	2.x	Data validation and schema enforcement for configs and API responses
PyYAML	6.x	YAML parsing for agents.yaml configuration files
Flask	2.x	HTTP backend for both Web UI and Agentic Team UI
Flask-SocketIO	4.x	WebSocket/Socket.IO for real-time streaming to Web UI
httpx	latest	Async HTTP client for Ollama and LlamaCpp REST API calls
tenacity	latest	Retry logic with exponential backoff decorators
prometheus-client	latest	Prometheus metrics exposition on /metrics endpoint
structlog	latest	Structured JSON logging with context binding
sentence-transformers	latest	all-MiniLM-L6-v2 embeddings for semantic context search

Frontend — Vue 3 Ecosystem

Library / Tool	Version	Role in System
Vue 3	3.x	Reactive UI framework with Composition API
Nuxt	3.x	SSR-capable meta-framework for the dashboard
Vite	5.x	Build tool — fast HMR, ES module bundling
Pinia	2.x	State management store — replaces Vuex
TailwindCSS	3.x	Utility-first CSS framework for rapid styling
Monaco Editor	latest	VS Code editor component for code display/editing in UI
Apollo Vue GraphQL	3.x	GraphQL client for structured data queries
Socket.IO Client	4.x	Real-time WebSocket communication with Flask backend
vis-network	9.x	Interactive graph visualization for Context Dashboard
Chart.js	4.x	Analytics charts in HTML reports and Context Dashboard

DevOps & Quality

Tool	Purpose
Docker / Docker Compose	Containerization; multi-service orchestration with optional monitoring profile

Tool	Purpose
Kubernetes	Production-grade horizontal scaling with manifests in deployment/kubernetes/
Prometheus + Grafana	Metrics scraping, dashboards, alerting
NGINX	Reverse proxy / load balancer for Web UI in production
GitHub Actions	CI/CD pipeline — lint, test, build, release on push to main
GitLab CI	Alternative CI pipeline via .gitlab-ci.yml
CircleCI	Additional CI configuration via .circleci/config.yml
Jenkins	Enterprise CI via Jenkinsfile
pytest + coverage	Unit, integration, shell tests; >80% coverage target
Black + isort	Code formatting — enforced by pre-commit hooks
Flake8 + mypy	Linting and static type checking
Bandit	Security scanning of Python code
pre-commit	15 hooks run automatically on every commit

SECTION 06

Folder and File Explanation

orchestrator/

Core orchestration engine. Contains all workflow, config, security, caching, retry, metrics, fallback, and offline detection logic. This is the heart of the system.

File / Subfolder	Description
core.py	Main OrchestratorCore class. Entry point for task execution. Coordinates all subsystems.
workflow.py	WorkflowEngine — loads YAML workflow definitions, manages step execution and iterations.
shell.py	Interactive CLI REPL. Manages conversation state, follow-up detection, history.
config_manager.py	ConfigManager — loads agents.yaml, validates agent/workflow definitions, exposes settings.
metrics.py	Prometheus metric definitions. Singleton collector initialized once at startup.
security.py	SecurityManager — input validation, command injection prevention, rate limiting.
cache.py	Two-tier cache (in-memory TTL=5min, file TTL=24h). Keyed by task+workflow hash.
retry.py	Retry decorators using tenacity with exponential backoff and jitter.
fallback.py	FallbackManager — maps cloud agents to local equivalents; invoked on agent failure.
offline.py	OfflineDetector — cached DNS/HTTP connectivity checks; filters non-local agents in offline mode.
observability/	ReportGenerator producing JSON and Chart.js HTML dashboard reports per task.
context/	Graph context memory: memory_manager.py, graph_store.py (SQLite+WAL+FTS5), BM25, embeddings, hybrid search, analytics, pruning, export, versioning.

adapters/

AI agent adapter implementations. Each adapter wraps a specific AI tool behind the BaseAdapter interface.

File / Subfolder	Description
base.py	Abstract BaseAdapter class. Defines the contract: get_capabilities(), execute_task(), execute_task_async(), is_available().
claude_adapter.py	Wraps Claude Code CLI subprocess. Handles authentication, prompt formatting, timeout.
codex_adapter.py	Wraps OpenAI Codex CLI. Manages API key injection and response parsing.
gemini_adapter.py	Wraps Google Gemini CLI. Handles auth and model selection.
copilot_adapter.py	Wraps GitHub Copilot CLI. Authentication via gh token, subprocess communication.
ollama_adapter.py	HTTP client for Ollama REST API. Supports list_models(), pull_model(), remove_model(), async execution.

File / Subfolder	Description
llama_cpp_adapter.py	HTTP client for any OpenAI-compatible local endpoint. Configurable endpoint URL and model name.
cli_communicator.py	Shared subprocess abstraction. Process spawning, stdout/stderr capture, timeout enforcement, error normalization.

agentic_team/

Standalone Agentic Team runtime. Completely independent of orchestrator/.

File / Subfolder	Description
engine.py	AgenticTeamEngine — drives the turn loop, manages role-to-adapter bindings, parses JSON decisions, emits callbacks.
shell.py	agentic-shell REPL. Accepts task input, invokes engine, displays turn-by-turn output.
__init__.py	Package init. Exposes AgenticTeamEngine for programmatic use.
context/	Independent copy of the graph context system tailored for the Agentic Team (uses FTS5 instead of sentence-transformers).

ui/

Web UI backends and frontend application.

File / Subfolder	Description
app.py	Flask backend for Orchestrator Web UI. REST endpoints + Socket.IO events for real-time streaming.
agentic_app.py	Flask backend for Agentic Team UI. Exposes /api/execute and emits team_turn, team_communication, progress_log events.
frontend/	Vue 3 / Nuxt frontend application with Pinia stores, Vite build, TailwindCSS, Monaco editor.
static/	Static assets for the standalone Agentic Team HTML UI.
templates/	Jinja2 templates including agentic_team.html for the team turn visualization.
requirements.txt	UI-specific Python dependencies (Flask, Flask-SocketIO, etc.).

config/

System configuration files.

File / Subfolder	Description
agents.yaml	Master configuration: agent definitions (name, type, command/endpoint, model, offline flag, enabled), workflow step sequences, global settings (max_iterations, fallback map, offline detection, report generation).

tests/

Comprehensive test suite (386 tests).

File / Subfolder	Description
test_orchestrator.py	Unit tests for OrchestratorCore, ConfigManager, FallbackManager, OfflineDetector.

File / Subfolder	Description
test_adapters.py	Unit tests for all six adapters with mock subprocess and HTTP responses.
test_integration.py	Integration tests: full task execution through a mock workflow.
test_shell.py	Shell REPL tests: follow-up detection, session save/load, command parsing.

deployment/

Production deployment configurations.

File / Subfolder	Description
kubernetes/	K8s manifests: Deployments, Services, Ingress, PersistentVolumeClaims, Namespace, ConfigMaps.
systemd/	systemd service unit files for bare-metal Linux deployment.

.circleci/, .github/, .gitlab-ci.yml, Jenkinsfile

CI/CD pipeline configurations for CircleCI, GitHub Actions, GitLab CI, and Jenkins respectively. Pipelines run lint → type-check → security scan → tests → build → release.

.claude/, .codex/

Agentic infrastructure configuration for Claude Code and Codex CLI tools.

File / Subfolder	Description
.claude/agents/	11 Markdown role-definition files for Claude Code sub-agents (web-frontend, backend-api, security-specialist, etc.).
.claude/skills/	22 skill documents providing domain knowledge auto-injected into agent prompts.
.claude/rules/	11 domain rule files defining mandatory coding standards always enforced by Claude.
.codex/agents/	13 TOML role-definition files for Codex CLI agents.
AGENTS.md	Universal agent instructions read by Codex, Gemini CLI, and other agentic tools.

graphify/

Standalone code knowledge graph engine (zero imports from orchestrator or agentic_team). Builds deep AST-based graphs from project directories using 6 language analyzers, 15 node types, 11 edge types, and 23 language detections. Supports file watching, graph snapshots, and a REST API.

reports/, sessions/, workspace/

Runtime-generated directories. reports/ holds JSON and HTML dashboard files. sessions/ persists conversation state. workspace/ is the default working directory for agent-generated files.

scripts/

Utility scripts including seed_context_graphs.py for pre-populating context databases with sample data.

context_dashboard/

Standalone Flask web application (port 5003) providing an interactive vis.js graph explorer and Chart.js analytics dashboard aggregating both context databases.

examples/

Usage examples demonstrating common workflow patterns and programmatic API usage.

packages/

Shared Python packages used across modules (utilities, common types, constants).

docs/images/

Screenshots and diagrams for documentation (cli.png, ui.png, agentic-team.png, etc.).

SECTION 07

Agents Deep Dive

7.1 Cloud AI Adapters

Agent	Adapter File	Communication Method	Key Characteristics
Claude	claude_adapter.py	Subprocess — claude CLI	Best at reasoning, documentation, and nuanced code refinement. Requires Claude Code CLI authenticated in the venv.
Codex	codex_adapter.py	Subprocess — codex CLI	Optimized for initial code generation and implementation. Requires OpenAI API key.
Gemini	gemini_adapter.py	Subprocess — gemini CLI	Strong at code review, analysis, and multi-turn conversation. Requires Google Cloud auth.
Copilot	copilot_adapter.py	Subprocess — gh copilot CLI	Inline suggestion and alternative approach generation. Requires GitHub Copilot subscription.

7.2 Local AI Adapters

Agent	Adapter File	Communication Method	Key Characteristics
Ollama	ollama_adapter.py	HTTP POST to /api/generate (localhost:11434)	Supports any Ollama-managed model (codellama, llama3, mistral, etc.). Full model management: list, pull, remove. Fully async.
LlamaCpp	llama_cpp_adapter.py	HTTP POST to OpenAI-compatible /v1/chat (configurable endpoint)	Works with llama.cpp server, LM Studio, text-generation-webui, or any OpenAI-compatible local API. Configurable model name and endpoint URL.

7.3 Agentic Team Roles

In Agentic Team mode, each AI agent is assigned a named role. The role binding is configured in agents.yaml under the agentic_team.roles key. Each role can be bound to any available adapter.

Role	Default Binding	Responsibilities
Project Manager (Lead)	Claude	Receives initial task, delegates to team, gates final output with 'finalize' action
Software Architect	Gemini	High-level design, technology selection, architectural decisions
Software Developer	Codex	Implementation, code generation, feature development
QA Engineer	Gemini	Test planning, code review, quality gates, bug identification
DevOps Engineer	Codex	Deployment strategy, CI/CD pipeline design, infrastructure considerations

7.4 Specialized Claude Sub-Agents

The .claude/agents/ directory contains 11 specialized personas invoked via @agent-name in Claude Code. These are not separate processes but system-prompt-based role switches within Claude Code. Examples:

- **@security-specialist** — OWASP top-10 expertise, vulnerability analysis, secure coding standards
- **@ai-ml-engineer** — ML pipelines, LLM integration, RAG architectures, embeddings, fine-tuning
- **@devops-infrastructure** — Docker multi-stage builds, Kubernetes manifests, CI/CD pipeline design
- **@performance-engineer** — Profiling, caching strategies, load testing, async optimization
- **@database-architect** — Schema design, query optimization, migration strategies, indexing

SECTION 08

Execution Flow (Step-by-Step)

8.1 Orchestrator Workflow Execution

Step 1. User Input

User submits a task via CLI (./ai-orchestrator run 'task') or Web UI POST request.

Step 2. Input Validation

SecurityManager validates input: checks for command injection, path traversal, and malicious payloads. Raises SecurityError on violation.

Step 3. Config Resolution

ConfigManager loads config/agents.yaml. Resolves selected workflow (or uses default). Validates all referenced agents are defined.

Step 4. Mode Detection

OfflineDetector checks --offline flag, settings.offline.enabled, and cached connectivity. Sets online/offline mode. In offline mode, non-local adapters are removed from the step list.

Step 5. Adapter Init

For each agent in the workflow, the appropriate adapter class is instantiated based on agents..type field (cli, ollama, llamacpp).

Step 6. Cache Check

CacheLayer checks if an identical task+workflow result is cached within TTL. Returns cached result if hit.

Step 7. Workflow Execution

WorkflowEngine iterates over workflow steps. For each step: (a) builds context from previous steps; (b) calls adapter.execute_task(task, context); (c) on failure, tries fallback adapter via FallbackManager; (d) appends result to context.

Step 8. Result Aggregation

Orchestrator collects all step results. Identifies the final refined output (last step). Extracts code blocks and maps them to file paths.

Step 9. File Writing

Detected files are validated, backed up if existing, and written to workspace/. File registry is updated.

Step 10. Session Update

Conversation context (task + result) stored in session. Follow-up detection state updated.

Step 11. Report Generation

If create_reports=true, ReportGenerator writes JSON execution summary, agent performance, workflow analytics, system health, config audit, and HTML dashboard to reports/.

Step 12. Metrics Update

Prometheus counters/histograms updated: task count, duration, agent call counts, error counts, cache hits.

Step 13. Response Return

Final output returned to CLI (formatted with Rich) or to Web UI (emitted via Socket.IO task_completed event).

8.2 Agentic Team Execution Flow

Step 1. Task Submission

User submits task via agentic-shell REPL or HTTP POST /api/execute to agentic_app.py.

Step 2. Validation

AgenticTeamEngine validates role-to-adapter mappings. Rejects run if any role has no available bound adapter.

Step 3. Lead Initialization

Engine generates initial prompt for the Lead role (Project Manager): task description + team roster + empty transcript.

Step 4. Turn Loop

Engine calls adapter.execute_task(role_prompt) for the current role's bound adapter. Expects a JSON response with fields: action ('message' or 'finalize'), to_role, message.

Step 5. Decision Parsing

Engine parses and normalizes the JSON decision. Validates action and to_role fields. Falls back to default routing on malformed responses.

Step 6. Event Emission

Engine calls turn_callback with step payload. UI backend emits Socket.IO events: team_turn (timeline), team_communication (graph edge), progress_log (console panel).

Step 7. Routing

If action='message': engine sets to_role as the next active role and appends the message to the transcript. Loop repeats.

Step 8. Termination

If action='finalize' AND current role is the lead: engine returns final output. If max_turns is reached: engine returns bounded fallback output of the last lead message.

SECTION 09

Configuration System

The entire system is driven by config/agents.yaml — a single, human-readable YAML file that defines all agents, workflows, and runtime settings. The ConfigManager (orchestrator/config_manager.py) loads and validates this file at startup using Pydantic schemas.

9.1 Agent Configuration Schema

Field	Type	Required	Description
type	str	Yes	Adapter type: 'cli', 'ollama', or 'llamacpp'
command	str	CLI only	CLI executable name (e.g., 'claude', 'codex', 'gemini')
endpoint	str	Local only	HTTP endpoint URL for Ollama/LlamaCpp (e.g., http://localhost:11434)
model	str	Local only	Model name for local adapters (e.g., 'codellama:13b')
offline	bool	No	Mark agent as local/offline-capable. Defaults: Ollama=true, LlamaCpp=true, CLI=false
enabled	bool	No	Whether agent is available for workflow assignment. Default: true
timeout	int	No	Execution timeout in seconds. Default: 300

9.2 Workflow Configuration Schema

Workflows can be defined in two forms: a simple list of agent names (shorthand) or a full step definition with role, description, and optional fallback. Supported step fields:

Field	Description
agent	Agent name (must match a key in the agents section)
role	Step role label: 'implementer', 'reviewer', 'refiner', 'documenter', etc.
task	Alternative to role — legacy field for backward compatibility
description	Human-readable description of this step's purpose
fallback	Agent name to use if this step's primary agent fails (step-level fallback)

9.3 Global Settings

Setting Key	Default	Description
max_iterations	5	Maximum workflow iterations before forced termination
fallback.enabled	true	Enable cloud-to-local fallback routing
fallback.map	{}	Explicit agent → fallback mappings (e.g., claude: local-instruct)
offline.enabled	false	Force offline mode regardless of connectivity
offline.auto_detect	true	Auto-detect connectivity and switch to offline if unreachable
create_reports	false	Auto-generate execution reports to reports/ directory

Setting Key	Default	Description
project_path	null	Root path for project-scoped context graph partitioning

9.4 Environment Variables

Variable	Purpose
ANTHROPIC_API_KEY	API key for Claude (used by Claude Code CLI)
OPENAI_API_KEY	API key for Codex (used by OpenAI Codex CLI)
GOOGLE_API_KEY	API key for Gemini CLI
GITHUB_TOKEN	GitHub token for Copilot CLI authentication
LOG_LEVEL	Logging verbosity (DEBUG, INFO, WARNING, ERROR)
ENABLE_METRICS	Enable/disable Prometheus metrics exposition (true/false)
PROJECT_PATH	Override project path for context graph scoping

SECTION 10

Observability and Reporting

10.1 Prometheus Metrics

The MetricsCollector (orchestrator/metrics.py) initializes a singleton Prometheus registry and exposes a /metrics endpoint on port 9090. Metrics are updated throughout task execution.

Metric Name	Type	Labels	Description
orchestrator_tasks_total	Counter	workflow, status	Total tasks executed by workflow and status
orchestrator_task_duration_seconds	Histogram	workflow	Task execution time distribution
orchestrator_task_failures_total	Counter	workflow, reason	Task failures by workflow and failure reason
orchestrator_agent_calls_total	Counter	agent, role	Per-agent invocation counts by role
orchestrator_agent_errors_total	Counter	agent, error_type	Agent-specific error counts by type
orchestrator_agent_response_time_seconds	Histogram	agent	Per-agent response time distribution
orchestrator_cache_hits_total	Counter	—	Cache hit count
orchestrator_cache_misses_total	Counter	—	Cache miss count
orchestrator_active_sessions	Gauge	—	Currently active conversation sessions

10.2 Report Generation

When create_reports=true, the ReportGenerator automatically produces six report artifacts per task execution:

Report Type	Filename Pattern	Contents
Execution Summary	exec_*.json	Task ID, workflow, steps executed, agents used, fallbacks triggered, suggestions count, total duration
Agent Performance	perf_*.json	Per-agent: success rate, total calls, error breakdown, task type distribution
Workflow Analytics	workflow_*.json	Per-workflow: run count, success rate, average iterations, average duration
System Health	health_*.json	Disk usage, memory, Python version, platform, agent availability status
Config Audit	config_*.json	Active agents, workflow definitions, settings snapshot, enabled state
HTML Dashboard	dashboard_*.html	Interactive Chart.js dashboard: KPI cards, daily volume bar chart, agent success/failure stacked bar, duration trend line, workflow distribution doughnut
Report Catalog	INDEX.json	Index of all generated reports with timestamps and types

10.3 Health Checks

The Flask Web UI backend exposes two health check endpoints suitable for Kubernetes probes:

- **/health** — Overall health status. Returns 200 if orchestrator core is initialized and config is loaded.
- **/ready** — Readiness probe. Returns 200 only if at least one agent is available and responsive.

10.4 Structured Logging

All modules use structlog for structured JSON logging. Log entries include: timestamp, log level, module name, task_id, workflow, agent, duration_ms, and success flag. This format is compatible with log aggregation stacks (ELK, Loki, CloudWatch). The LOG_LEVEL environment variable controls verbosity globally.

SECTION 11

Key Concepts and Learnings

Multi-Agent Orchestration

The practice of coordinating multiple AI models in a pipeline where each agent's output becomes the next agent's input. This enables specialization: one model generates, another reviews, a third refines. The system demonstrates that heterogeneous AI collaboration produces better outcomes than single-model approaches for complex tasks.

Adapter Pattern in AI Systems

By defining a BaseAdapter interface, the system achieves full substitutability between cloud CLI tools and local HTTP APIs. New AI tools can be integrated by implementing five methods — no changes to the orchestration layer required. This pattern is critical for system longevity as the AI tooling landscape evolves rapidly.

Agentic vs. Orchestrated AI

The repository distinguishes two paradigms: Orchestrated AI (deterministic pipelines where the orchestrator controls agent sequencing) versus Agentic AI (agents make their own routing decisions in a free-communication model). Both paradigms are valid for different use cases — orchestrated for predictable workflows, agentic for exploratory or collaborative problem-solving.

Cloud-to-Local Fallback

A resilience pattern where cloud AI agents (Claude, Codex, Gemini) are paired with local model equivalents (Ollama, LlamaCpp). The FallbackManager automatically routes to local models when cloud agents fail or network is unavailable. This ensures uninterrupted development even in offline or air-gapped environments.

Persistent Graph Memory

Unlike stateless AI interactions, this system builds a persistent knowledge graph from all interactions — tasks, mistakes, patterns, decisions, preferences. Using BM25 + semantic (sentence-transformer) hybrid search with Reciprocal Rank Fusion, agents can retrieve relevant past context, learn from mistakes, and avoid repeating errors across sessions.

Production Readiness as a First-Class Concern

The codebase achieves Pylint 10.00/10 by enforcing zero-warning standards across 520 issues. This demonstrates that AI-assisted projects can and should meet the same code quality bars as traditional software. The pre-commit hooks, type checking, security scanning, and >80% test coverage are non-negotiable quality gates.

CLI-First Architecture

Building CLI tools with subprocess communication (rather than SDK integration) provides model-agnosticism — the system works with any AI tool that exposes a CLI interface, regardless of its internal API. This design decision trades some performance for extreme flexibility and future-proofing.

Context Window Management in Multi-Step Workflows

Each workflow step accumulates context from prior steps. The system must carefully manage what context to pass forward — too little loses continuity, too much exceeds model context windows. The current implementation passes a rolling summary of previous step outputs.

SECTION 12

Simplified Explanation for Beginners

Imagine you are a manager at a company. You have three specialist employees: Alice who is great at writing first drafts, Bob who is an expert code reviewer, and Carol who is excellent at polishing and documenting. When a new project comes in, you don't give it to just one person. Instead, you send it to Alice first to write a draft, then Bob reviews it and suggests improvements, then Carol polishes the final version. This is exactly what the AI Agents Orchestrator does — except the 'employees' are AI models (Claude, Codex, Gemini, Copilot) and the 'manager' is the orchestrator software.

The Three Big Ideas

1. Workflows are Assembly Lines

Just like a car factory has stations for welding, painting, and quality control, the orchestrator has workflow 'stations' for coding, reviewing, and refining. You can define your own assembly line in a YAML file.

2. Adapters are Universal Power Sockets

Each AI tool (Claude, Ollama, Codex) speaks a slightly different 'language'. Adapters are like travel adapter plugs — they let all these different tools plug into the same orchestrator regardless of their individual quirks.

3. The Agentic Team is a Virtual Meeting Room

Beyond the assembly line, there is also a 'meeting room' mode where AI agents take on job titles (Project Manager, Developer, QA Engineer) and have a free conversation to solve a problem together. The Project Manager chair is the only one who can officially close the meeting with a final answer.

What Happens When You Type a Command

What You Type	What Happens Behind the Scenes
./ai-orchestrator shell	Starts a REPL terminal where you can type tasks in plain English, like chatting with an AI assistant.
./ai-orchestrator run 'Build a REST API'	The orchestrator picks the 'default' workflow (Codex → Gemini → Claude), runs each AI agent in sequence, and returns the final polished code.
./ai-orchestrator agentic-shell	Opens a team meeting room where PM, Architect, Developer, QA, and DevOps agents collaborate on your task.
./ai-orchestrator run 'task' --offline	Skips all cloud AI tools and routes everything to your local Ollama or LlamaCpp server instead.

SECTION 13

Interview Questions and Answers

13.1 Beginner Level (15 Questions)

1. What is the AI Agents Orchestrator?

It is a production-ready platform that coordinates multiple AI coding assistants (Claude, Codex, Gemini, Copilot, Ollama) in configurable workflows to produce higher-quality code than any single model could alone. It exposes both a CLI REPL and a Vue 3 web dashboard.

2. What programming languages are used?

The backend is Python (57.2%), the web frontend is Vue 3 / JavaScript (12.8% JS + 6.8% Vue), with HTML (9.8%), CSS (5.3%), and Shell scripts (4.2%). Python is primary for all orchestration logic.

3. What is a workflow in this system?

A workflow is a named, ordered sequence of agent steps defined in config/agents.yaml. For example, the 'default' workflow chains Codex (implementation) → Gemini (review) → Claude (refinement).

4. What is the difference between the CLI and the Web UI?

The CLI (Click + Rich) is a terminal-based REPL suitable for developers who prefer the command line. The Web UI (Vue 3 + Flask + Socket.IO) adds a visual Monaco code editor, real-time streaming, and file download capabilities for a more graphical experience.

5. What is an adapter in this system?

An adapter is a Python class that wraps a specific AI tool behind a common interface (BaseAdapter). It translates the orchestrator's generic execute_task() calls into the specific subprocess commands or HTTP requests that each AI tool requires.

6. What is agents.yaml used for?

It is the master configuration file that defines every AI agent (name, type, command/endpoint, model, enabled flag), every workflow (step sequences), and global settings (max iterations, fallback map, offline mode, report generation).

7. How do you run the system offline?

Pass the --offline flag: ./ai-orchestrator run 'task' --offline. The system will skip all cloud CLI agents and route only to local Ollama or LlamaCpp adapters. Auto-detection can also trigger offline mode when internet connectivity is unavailable.

8. What is Prometheus used for here?

Prometheus collects metrics about system performance — how many tasks ran, how long they took, how many errors occurred, and cache performance. These metrics are exposed on :9090/metrics and can be visualized in a Grafana dashboard.

9. What does the session manager do?

It saves and restores conversation context between CLI sessions. Users can type /save session-name to persist the current conversation and /load session-name to continue a previous one. Sessions are stored as JSON files in the sessions/ directory.

10. What testing framework is used?

pytest with the coverage plugin. The test suite contains 386 tests across four files: test_orchestrator.py, test_adapters.py, test_integration.py, and test_shell.py. Coverage target is >80%.

11. What is the Agentic Team mode?

A separate runtime (agentic_team/) where AI agents take named roles (Project Manager, Software Architect, Developer, QA Engineer, DevOps Engineer) and communicate freely turn-by-turn. The lead role gates the final response.

12. How do you add a new AI agent?

Create a new adapter file in adapters/ extending BaseAdapter, implement the required methods, add the agent definition to config/agents.yaml with the appropriate type field, and reference it in a workflow.

13. What is Docker used for in this project?

Docker containerizes the orchestrator for reproducible, environment-agnostic deployment. A Dockerfile and docker-compose.yml are provided. The compose file includes an optional monitoring profile that adds Prometheus and Grafana services.

14. What is a fallback agent?

A fallback agent is an alternative AI model invoked when the primary agent fails. The FallbackManager maps cloud agents to local equivalents (e.g., claude → local-instruct) and automatically retries with the fallback on recoverable failures.

15. What is Rich used for?

Rich is a Python terminal formatting library used by the CLI to render colored text, progress bars, formatted tables, and panel layouts in the terminal. It makes the CLI output visually readable and professional.

13.2 Intermediate Level (15 Questions)

1. Explain the Adapter design pattern as implemented here.

BaseAdapter (adapters/base.py) defines an abstract interface with four methods: get_capabilities(), execute_task(), execute_task_async(), and is_available(). Each concrete adapter (ClaudeAdapter, OllamaAdapter, etc.) implements this interface. The orchestration layer only knows about BaseAdapter — it never directly references concrete adapter classes. New agents can be added without changing the orchestrator core.

2. How does the fallback mechanism work?

The FallbackManager reads the fallback.map in agents.yaml (e.g., claude: local-instruct). When a workflow step's adapter raises a recoverable exception (network timeout, auth error), the FallbackManager is invoked. It looks up the fallback mapping for that agent name, instantiates the fallback adapter, and re-executes the same step. Non-recoverable errors (e.g., malformed task) are not retried.

3. Describe the offline detection system.

OfflineDetector (orchestrator/offline.py) first checks the --offline CLI flag and the settings.offline.enabled config value. If neither forces offline mode and auto_detect=true, it performs a cached DNS/HTTP connectivity check (cached to avoid repeated latency). In offline mode, the adapter initialization step filters out all agents where offline=false, ensuring only local adapters (Ollama, LlamaCpp) participate in workflow execution.

4. How is caching implemented and what are its tiers?

The CacheLayer (orchestrator/cache.py) implements a two-tier cache. Tier 1 is an in-memory dictionary with a 5-minute TTL, keyed by a hash of the task string and workflow name. Tier 2 is a file-based cache (pickled or JSON) with a 24-hour TTL stored in a temp directory. Cache hits return the stored AgentResponse immediately without invoking any AI agent. An optional Redis tier is mentioned for distributed deployments.

5. What are the Socket.IO events emitted during Agentic Team execution?

Three events are emitted per turn via turn_callback: (1) team_turn — contains turn number, from_role, to_role, from_agent, to_agent, action, and message for the timeline view; (2) team_communication — aggregated directed edge data (from_role, to_role, count, latest, selected) for the graph visualization; (3) progress_log — human-readable log entry for the runtime log panel. A final task_completed event is emitted when the engine returns.

6. How does the retry logic work?

orchestrator/retry.py uses the tenacity library to implement exponential backoff with jitter. The @with_retry(max_attempts=3) decorator wraps adapter execute_task calls. Tenacity retries on specific exception types (network errors, timeouts), waiting min=2s to max=10s between attempts with a multiplier of 1. Permanent failures (security errors, invalid inputs) are not retried.

7. Explain the Graph Context Memory architecture.

Each runtime (Orchestrator and Agentic Team) maintains an independent SQLite database with WAL mode enabled. Nodes represent 10 types of knowledge entities (task, mistake, pattern, decision, etc.) with optional sentence-transformer embeddings. Edges represent 12 semantic relationships. Search supports three modes: BM25 keyword (bm25_index.py), semantic cosine similarity (embeddings.py using all-MiniLM-L6-v2), and hybrid (hybrid_search.py using Reciprocal Rank Fusion with k=60). Results are surfaced to agents as prior context for new tasks.

8. What is Reciprocal Rank Fusion and why is it used here?

RRF is a technique for combining ranked lists from multiple search methods. Each result's score is computed as sum(1 / (k + rank)) across all lists, where k=60 is a smoothing constant. This merges BM25 keyword rankings with semantic similarity rankings without requiring score normalization. It is used here because keyword search excels at exact term matching (code identifiers, function names) while semantic search excels at conceptual similarity (same idea expressed differently).

9. How are project-scoped context graphs implemented?

When a PROJECT_PATH environment variable or settings.project_path config key is set, the MemoryManager computes a project_id as the first 16 characters of the SHA-256 hash of the normalized absolute path. All nodes created during that session are tagged with this project_id. Queries filter by project_id, ensuring knowledge from different projects does not bleed together. Global nodes (project_id='') are accessible from all projects.

10. Describe how the Web UI achieves real-time updates.

The Flask backend uses Flask-SocketIO to maintain WebSocket connections with the Vue 3 frontend. When a task is submitted via HTTP POST, the backend starts execution in a background thread (or async task). As each workflow step completes, the backend emits Socket.IO events (progress_update, step_complete, task_completed). The Vue 3 frontend listens to these events and reactively updates the Pinia store, which triggers component re-renders without polling.

11. What security controls are implemented?

The SecurityManager implements: (1) command injection prevention — detects shell metacharacters in user input; (2) path traversal protection — validates file paths against the workspace root; (3) rate limiting — token bucket algorithm per user session; (4) input sanitization — strips potentially harmful content before passing to AI agents; (5) audit logging — all security events logged with structlog; (6) environment-based secret management — no hardcoded credentials.

12. How does the system handle concurrent sessions?

Each CLI shell or Web UI connection maintains an independent session context (separate task history, separate file registry). The SessionManager assigns unique session IDs. Sessions are stored as separate JSON files in sessions/. The Prometheus active_sessions gauge tracks concurrent session count. The orchestrator core itself is stateless between calls — all state is in the session context.

13. What is the Graphify system and how does it differ from the context memory?

Graphify (graphify/) is a standalone code knowledge graph engine that analyzes a project directory using AST parsing across 6 language analyzers, producing a detailed graph of classes, functions, imports, dependencies, and documentation. It operates at the code structure level. The context memory system operates at the interaction/knowledge level (tasks, mistakes, preferences). Both systems are independent with zero cross-imports.

14. How is the CI/CD pipeline structured?

The project supports four CI platforms: GitHub Actions (.github/), GitLab CI (.gitlab-ci.yml), CircleCI (.circleci/), and Jenkins (Jenkinsfile). Each pipeline runs the same stages: lint (flake8, isort) → type-check (mypy) → security scan (bandit) → unit tests (pytest) → integration tests → build (Docker image) → release (tag-triggered). The Makefile provides local equivalents of all pipeline stages.

15. What MCP tools are exposed and how?

An optional FastMCP 3.x server (mcp_server/) exposes 34+ tools via the Model Context Protocol, categorized as: Orchestrator tools (4, e.g., run_workflow, list_agents), Agentic Team tools (5, e.g., start_team_session, get_turn_history), and Shared tools (1, e.g., get_health_status). MCP clients (Claude Desktop, Claude Code, custom LLM agents) can invoke these tools without direct Python API access. This component is entirely optional — the core systems do not depend on it.

13.3 Advanced Level (15 Questions)

1. How would you scale this system horizontally for high-throughput production use?

Horizontal scaling requires externalizing all state: replace file-based session storage with Redis (session_manager → Redis client), replace in-memory cache with Redis cache (cache.py → Redis backend), replace SQLite context DBs with PostgreSQL + pgvector for embedding search. The orchestrator core is already stateless between requests. Deploy multiple orchestrator pods behind an NGINX ingress. Use a message queue (Celery + Redis or Kafka) for task distribution to avoid synchronous blocking on slow AI CLI tools. The Kubernetes manifests in deployment/kubernetes/ provide the starting scaffold.

2. Explain the architectural decision to keep Agentic Team fully isolated from Orchestrator.

The isolation is an intentional architectural boundary: the two systems have different execution models (deterministic pipeline vs. dynamic role-based communication), different termination conditions (iteration count vs. lead finalization), and different context schemas. Sharing code would create coupling that makes it harder to evolve either independently. The duplicate structure (R0801 pylint suppression) is explicitly acknowledged as intentional parallel architecture, not accidental duplication. This mirrors the microservices principle of 'shared nothing' boundaries.

3. How does the Hybrid Search with RRF work technically, and what are its trade-offs?

BM25 (bm25_index.py) builds an inverted index over node content. For a query, it returns ranked results by TF-IDF score. Separately, embeddings.py generates a query embedding using all-MiniLM-L6-v2 (384-dimensional vectors) and computes cosine similarity against stored node embeddings. RRF fusion assigns each document a score of sum(1/(60+rank_i)) across both ranked lists. The k=60 constant de-emphasizes rank differences at the top. Trade-offs: sentence-transformer inference adds ~50-100ms per query; embeddings consume ~1.5KB per node; semantic search degrades on out-of-vocabulary terms. The Agentic Team context avoids these costs by using SQLite FTS5 only.

4. How would you add streaming output support to the CLI?

Currently, adapters return a complete AgentResponse after the subprocess/HTTP call finishes. To support streaming: (1) Add execute_task_stream() to BaseAdapter returning an AsyncGenerator[str, None]. (2) For CLI adapters, use subprocess.Popen with stdout=PIPE and read line-by-line in an async loop. (3) For Ollama/LlamaCpp, switch from stream=False to stream=True in the API request and parse server-sent events. (4) Update the CLI shell to use Rich's Live context for incremental rendering. (5) Update Socket.IO events to emit chunk events instead of a single task_completed.

5. What are the failure modes of the CLICommunicator and how are they handled?

CLICommunicator (adapters/cli_communicator.py) handles: (1) Process timeout — subprocess is killed after timeout seconds, TimeoutError raised, caught by retry logic; (2) Non-zero exit code — stderr is captured and included in the AgentResponse.error field; (3) Empty output — detected and treated as a soft failure, triggering fallback; (4) Authentication errors — detected by pattern-matching stderr output (e.g., 'not authenticated'), raised as a non-retryable exception; (5) CLI tool not found — FileNotFoundError raised, agent is_available() returns False, agent is removed from workflow.

6. Describe the Graphify AST analysis pipeline for Python files.

graphify/analyzers/python_analyzer.py uses Python's built-in ast module to parse source files. It walks the AST to extract: (1) CLASS nodes from ast.ClassDef — captures name, methods, base classes, decorators; (2) FUNCTION nodes from ast.FunctionDef/AsyncFunctionDef — captures name, args, return type annotation, decorators; (3) IMPORT nodes from ast.Import/ImportFrom — captures module paths for dependency edges; (4) VARIABLE nodes from ast.Assign at module scope. All nodes are content-hashed (SHA-256) for incremental scanning — only modified files are re-analyzed. CALLS edges are approximated via name matching (exact call graph requires runtime analysis).

7. How does project-scoped graph partitioning maintain data integrity during concurrent rescans?

The rescan_project() operation follows an atomic swap pattern: (1) delete_nodes_by_project() executes a single-transaction DELETE FROM nodes WHERE project_id=? to atomically remove all prior nodes; (2) register_project() rebuilds nodes in a new transaction. Between these two transactions there is a window where the project has no nodes — any concurrent read during this window returns empty results rather than stale data. The add_node() upsert uses INSERT ... ON CONFLICT(id) DO UPDATE SET (without cascade delete) to preserve edges when node content changes, avoiding orphaned edges.

8. How would you implement A/B testing of different workflow configurations?

Implement a WorkflowSelector that uses a hash of session_id modulo N to assign users to workflow variants. Store variant assignment in session metadata. Emit workflow variant as a Prometheus label (workflow_variant='A' or 'B') on all task metrics. After sufficient samples, compare: task_duration_seconds p50/p95, agent_errors_total, and qualitative output scores. The YAML-driven configuration makes it trivial to define variant workflows without code changes — add default_A and default_B workflow definitions and toggle via config.

9. What are the implications of using subprocess for CLI communication vs. SDK integration?

Subprocess approach: (1) Model-agnostic — works with any CLI tool regardless of SDK availability; (2) No SDK version conflicts — each tool manages its own dependencies; (3) Process overhead — each call spawns a new process (~50-200ms overhead); (4) No streaming — must wait for process completion; (5) Authentication complexity — each CLI manages auth independently; (6) Brittle output parsing — depends on CLI output format stability. SDK approach would offer better performance, streaming, and structured responses but at the cost of tight coupling to specific vendor SDK versions. The subprocess approach is correct for a framework that aims to support any AI tool, including those without Python SDKs.

10. Explain the versioning system in the Orchestrator context and its use cases.

ops/versioning.py implements a node history system: record_version(node_id, change_type) snapshots the node's current state (content, metadata, embedding, importance_score) before an update. Versions are stored in a node_versions table with version number, timestamp, and change_type. get_version(node_id, version) retrieves a specific snapshot. rollback(node_id, version) atomically replaces the current node state with the snapshot. diff_versions() computes a field-level diff between two versions. Use cases: (1) reverting to a better prior pattern after an experimental update; (2) auditing how a decision evolved over time; (3) implementing undo/redo for context graph edits.

11. How would you implement circuit breakers for AI agent calls?

Augment the FallbackManager with a per-agent CircuitBreaker state machine (CLOSED, OPEN, HALF_OPEN). On consecutive failures exceeding a threshold (e.g., 3 in 60 seconds), transition to OPEN state — all calls are immediately rejected and routed to fallback without attempting the primary agent. After a cooldown period (e.g., 5 minutes), transition to HALF_OPEN and allow one probe call. Success returns to CLOSED; failure returns to OPEN. Expose circuit state as a Prometheus gauge (orchestrator_circuit_state{agent='claude'}) for monitoring.

12. What changes are needed to support multi-modal tasks (image + code)?

Multi-modal support requires: (1) Extend AgentTask with an optional List[bytes] attachments field for image data; (2) Update BaseAdapter.execute_task() signature to accept attachments; (3) For CLI adapters, write image data to temp files and pass file paths as CLI arguments; (4) For Ollama/LlamaCpp adapters, base64-encode images and include in the images field of the API request (Ollama supports this for vision models like llava); (5) Update SecurityManager to validate image MIME types and size limits; (6) Update the Web UI to accept drag-and-drop file uploads and include them in task payloads.

13. How does the skills auto-activation mechanism work in the .claude/skills/ system?

Each skill document contains a front-matter section defining trigger_keywords, trigger_file_patterns, and trigger_task_types. Claude Code evaluates these triggers against the current task description and open files. When a match occurs, the skill document content is injected into the agent's system prompt as additional context. This is implemented at the Claude Code CLI level, not in the orchestrator Python code. The orchestrator's adapters pass the task description verbatim to the CLI — skill injection happens inside Claude Code's own context management.

14. Describe a strategy for making the context graph queryable across projects.

Currently, project_id='' (empty string) marks global nodes. A cross-project query layer would: (1) Allow the MemoryManager to accept an optional project_id=None parameter meaning 'all projects'; (2) Remove the WHERE project_id=? filter for cross-project searches; (3) Return nodes annotated with their project_id for provenance; (4) Implement a ProjectRegistry that maps project_ids to human-readable names; (5) Add a cross_project_search() method to HybridSearch that merges results across all project partitions with project_id as an additional ranking signal (favor results from the current project). This enables knowledge transfer — patterns learned in Project A become discoverable when working on a similar problem in Project B.

15. What are the key architectural improvements you would propose for v2.0?

Priority improvements: (1) Streaming output — replace request-response with SSE/WebSocket streams for real-time agent output; (2) Async-first core — rewrite orchestrator/core.py with asyncio throughout, enabling true parallel agent calls where workflow dependencies allow; (3) Distributed context storage — migrate SQLite to PostgreSQL + pgvector for multi-instance deployments; (4) Workflow as DAG — replace linear step lists with directed acyclic graphs enabling parallel branches (e.g., Codex and Gemini both review simultaneously, Claude synthesizes both reviews); (5) Agent health dashboard — real-time circuit breaker states, per-agent SLA monitoring, automatic agent pool management; (6) Semantic workflow selection — use the context graph to recommend the best workflow for a task based on similar past tasks.

SECTION 14

Mini Project Implementation Plan

This section outlines a 4-week implementation plan for building a simplified version of the AI Agents Orchestrator — suitable as a portfolio project, technical assessment submission, or interview demonstration piece. The plan covers all key concepts without requiring access to paid AI services (Ollama is used for local-only execution).

Project Goal

Build a minimal two-agent orchestrator: Agent A (Ollama with codellama) generates code, Agent B (Ollama with llama3) reviews it, and the system returns the reviewed output. Expose the system via a Click CLI and a simple Flask REST endpoint.

Week	Milestone	Deliverables	Skills Practiced
Week 1	Foundation	• Project scaffold (pyproject.toml, requirements.txt, .gitignore) • BaseAdapter abstract class with execute_task() and is_available() • OllamaAdapter implementing BaseAdapter using httpx • Basic pytest setup with mock HTTP responses	Adapter pattern, abstract classes, httpx, pytest mocking
Week 2	Orchestration Core	• agents.yaml schema with PyYAML + Pydantic validation • ConfigManager loading and validating agent/workflow definitions • WorkflowEngine executing a two-step sequence • Retry decorator with tenacity on adapter calls • Unit tests for ConfigManager and WorkflowEngine	YAML config, Pydantic, tenacity, workflow design
Week 3	CLI and Observability	• Click CLI with run and shell commands • Rich terminal formatting for output display • Simple in-memory caching (hash-keyed dictionary with TTL) • Prometheus counter/histogram metrics via prometheus-client • Structured logging with structlog • Integration test: full two-step workflow execution	Click, Rich, prometheus-client, structlog, integration testing
Week 4	Web UI and Polish	• Flask REST backend with POST /api/run endpoint • Flask-SocketIO for real-time step progress events • Minimal Vue 3 frontend with Vite consuming Socket.IO events • Docker Compose file running orchestrator + Prometheus • README with architecture diagram, setup instructions, and demo GIF • Final code quality pass: black, flake8, mypy, bandit	Flask, Socket.IO, Vue 3, Docker, documentation

File Structure for Mini Project

```
mini-orchestrator/
■■■ adapters/
■   ■■■ base.py            # BaseAdapter ABC
■   ■■■ ollama_adapter.py  # Ollama HTTP adapter
■■■ orchestrator/
■   ■■■ core.py            # OrchestratorCore
■   ■■■ workflow.py        # WorkflowEngine
■   ■■■ config_manager.py  # PyYAML + Pydantic config loader
■   ■■■ cache.py           # In-memory TTL cache
■   ■■■ retry.py           # tenacity retry decorator
■   ■■■ metrics.py         # Prometheus metrics
■■■ ui/
```

```
■   ■■■ app.py              # Flask + SocketIO backend
■   ■■■ frontend/           # Vue 3 + Vite
■■■ config/
■   ■■■ agents.yaml         # Agent + workflow definitions
■■■ tests/
■   ■■■ test_adapters.py    # Adapter unit tests
■   ■■■ test_workflow.py    # Workflow integration tests
■■■ ai-orchestrator         # Click CLI entry point
■■■ docker-compose.yml
■■■ requirements.txt
■■■ README.md
```

Stretch Goals (Beyond 4 Weeks)

- Add a third agent role (Agentic Team mode) with a simple turn loop and role-based prompting
- Implement a SQLite-backed session manager with /save and /load CLI commands
- Add a basic BM25 knowledge graph (skip sentence-transformers for simplicity)
- Deploy to a VPS with Docker Compose and expose the Web UI via NGINX reverse proxy
- Add GitHub Actions CI pipeline running lint → typecheck → tests on every push
- Implement Grafana dashboard consuming Prometheus metrics from the running orchestrator

■ Tip for interviews: Be prepared to explain why the Adapter pattern was chosen over direct SDK calls, why YAML-driven configuration is preferable to hard-coded workflows, and how you would evolve the system to support parallel agent execution using asyncio.gather(). These are the three most common follow-up questions for this type of system.

End of Report — AI Agents Orchestrator Technical Analysis — April 2026
Source Repository: https://github.com/hoangsonww/AI-Agents-Orchestrator